

deti
universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Processes and Inter-Process Communication

Real-Time Embedded Systems (#41505)

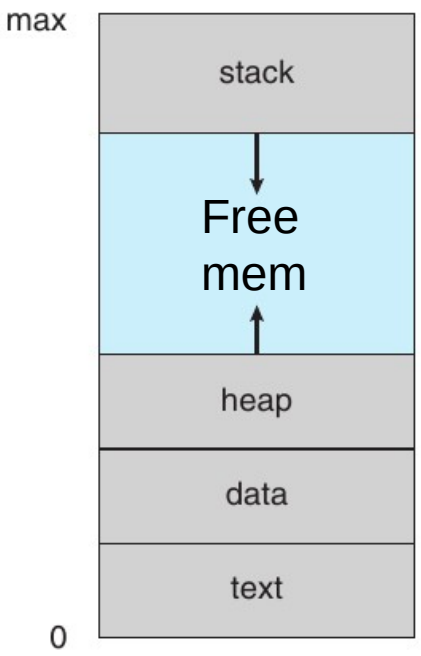
Paulo Pedreiras, pbrp@ua.pt
Department of Electronics, Telecommunications and Informatics
University of Aveiro
Academic year 2026/2027

Outline

- Process Concept
- Process Scheduling
- Operations on Processes
- Inter-process Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Examples of IPC Systems
- Communication in Client-Server Systems

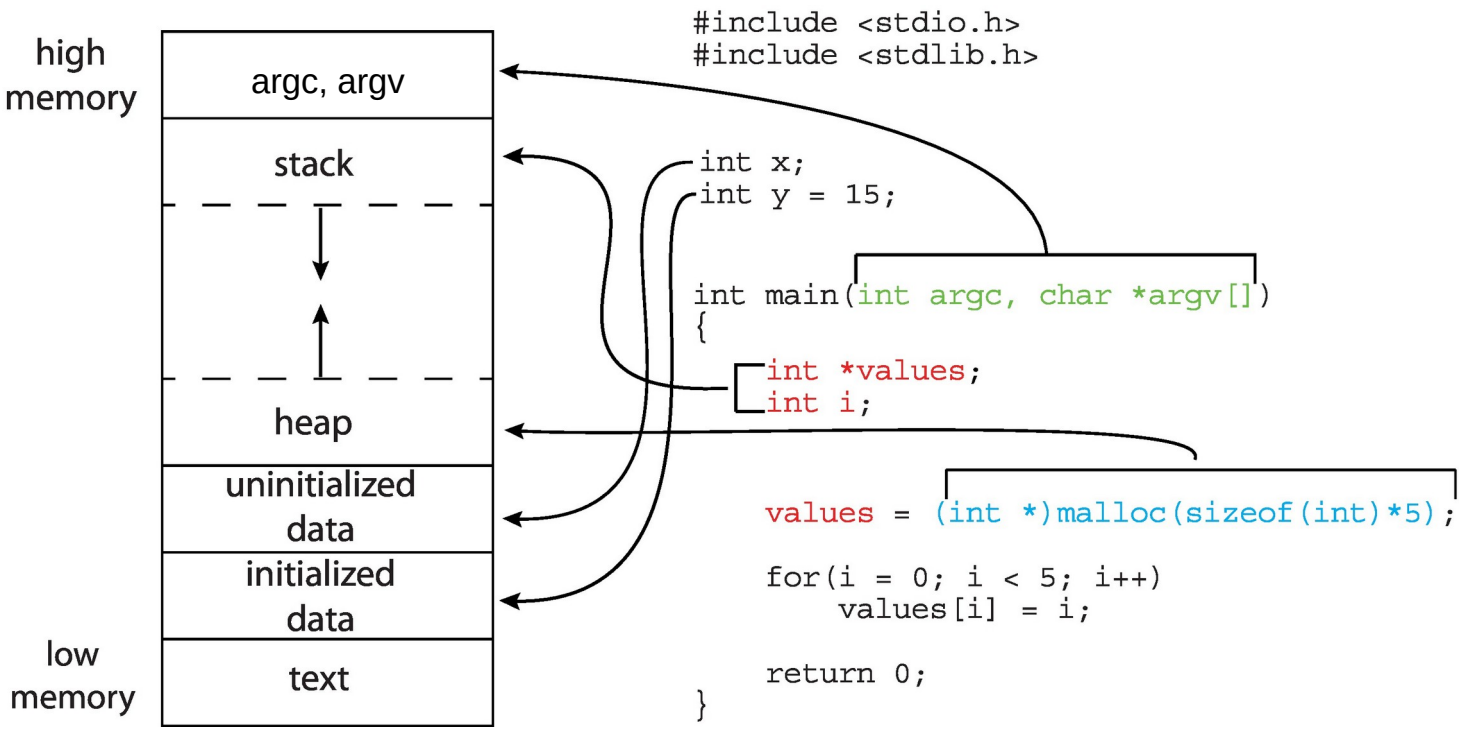
Process Concept

- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution;
- A process has multiple parts:
  - Current activity/state including **program counter**, processor registers, called **context**
  - The program code, also called **text section**
  - **Data section** containing global variables
  - **Stack** containing temporary data
    - Function parameters, return addresses, local variables
  - **Heap** containing memory dynamically allocated during run time



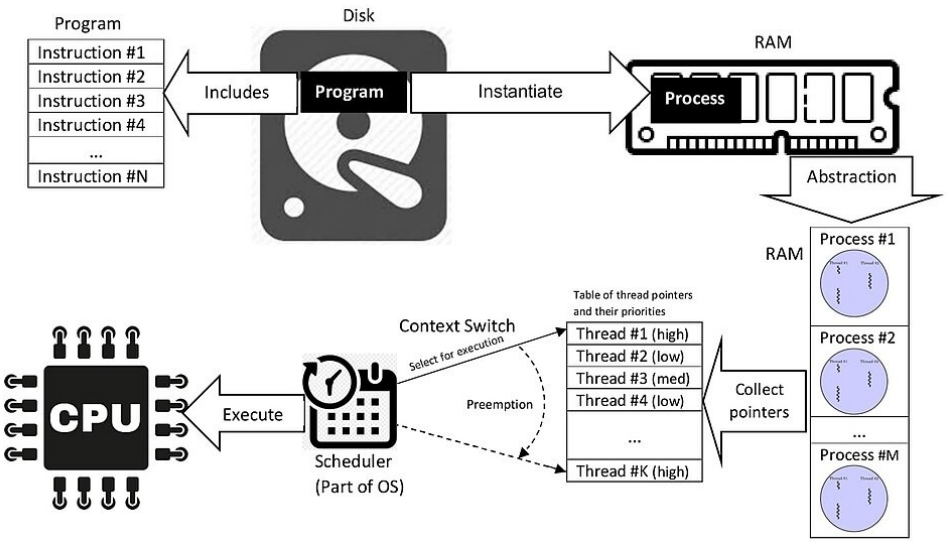
Process Concept

Memory layout



Process Concept

- A Program is **passive** entity stored on disk (**executable file**); a process is a matching **active** *entity*
  - **A program becomes process when an executable file is loaded into memory**

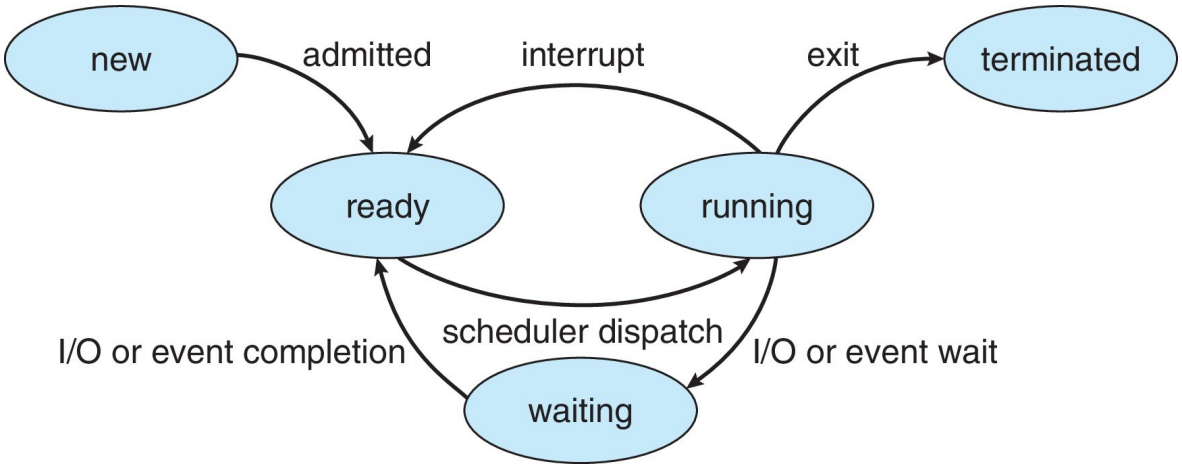


Source: https://commons.wikimedia.org/wiki/File:Concepts-_Program_vs._Process_vs._Thread.jpg

- The execution of program may be started via GUI mouse clicks, command line entry of its name, system daemons, etc.
- One single program can generate several processes executing at the same time:
  - Consider multiple users executing the same program, or a user running several instances of the same program (e.g. viewing two pdf files)

Process State

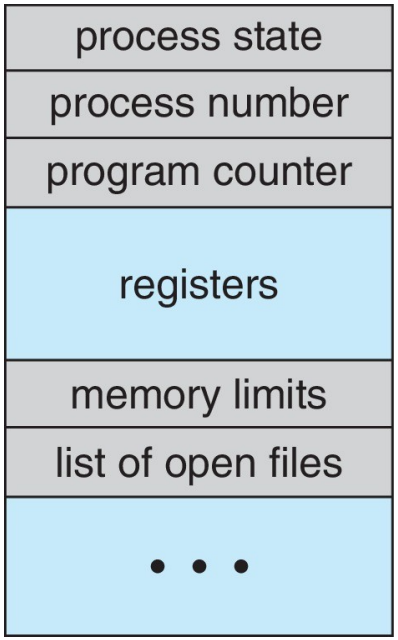
- As a process executes, it changes its **state**
  - **New**: The process is being created
  - **Running**: Instructions are being executed
  - **Waiting**: The process is waiting for some event to occur
  - **Ready**: The process is waiting to be assigned to a processor
  - **Terminated**: The process has finished execution



Process Control Block (PCB)

The PCB is a data structure that **holds all information** associated with each process, more commonly know as **Task Control Block – TCB**:

- **Process state** – running, waiting, etc.
- **Program counter** – location of next instruction to execute
- **CPU registers** – contents of all registers in use by the process
- CPU **scheduling information**- priorities, scheduling queue pointers
- **Memory-management information** – memory allocated to the process
- **Accounting information** – CPU used, clock time elapsed since start, time limits
- **I/O status information** – I/O devices allocated to process, list of open files



Threads

- The description so far assumes that a process has a single thread of execution
- Modern operating systems allow processes to have multiple concurrent threads of execution, or simply **threads**

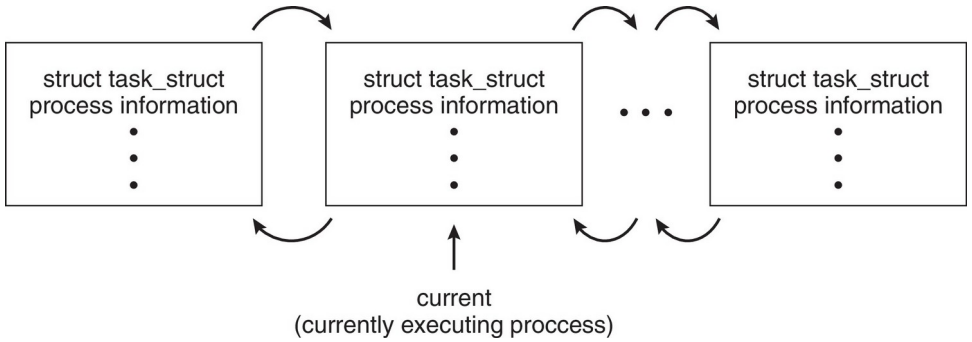
  E.g. a word processor can **run a spell checker while accepting user text**

- The TCB must then have storage for thread details (e.g. multiple program counters), ...
  - Topic covered in the next class. For the moment it will be assumed that a process only has one thread.

Process Representation in Linux

Represented by the C structure `task_struct`

```
pid t_pid;                      /* process identifier */
long state;                     /* state of the process */
unsigned int time_slice         /* scheduling information */
struct task_struct *parent;/* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files;/* list of open files */
struct mm_struct *mm;  /* address space of this process */
```



Process Scheduling

- The **scheduler** is the OS service responsible for selecting, among the (ready) processes, which process should be running at any instant
- The scheduler goal depends on the application scope:

  For GPOS: maximize CPU use, quickly switch processes onto CPU core, maximize throughput, ...

  For RTOS: assure **predictable and timely execution of critical tasks**

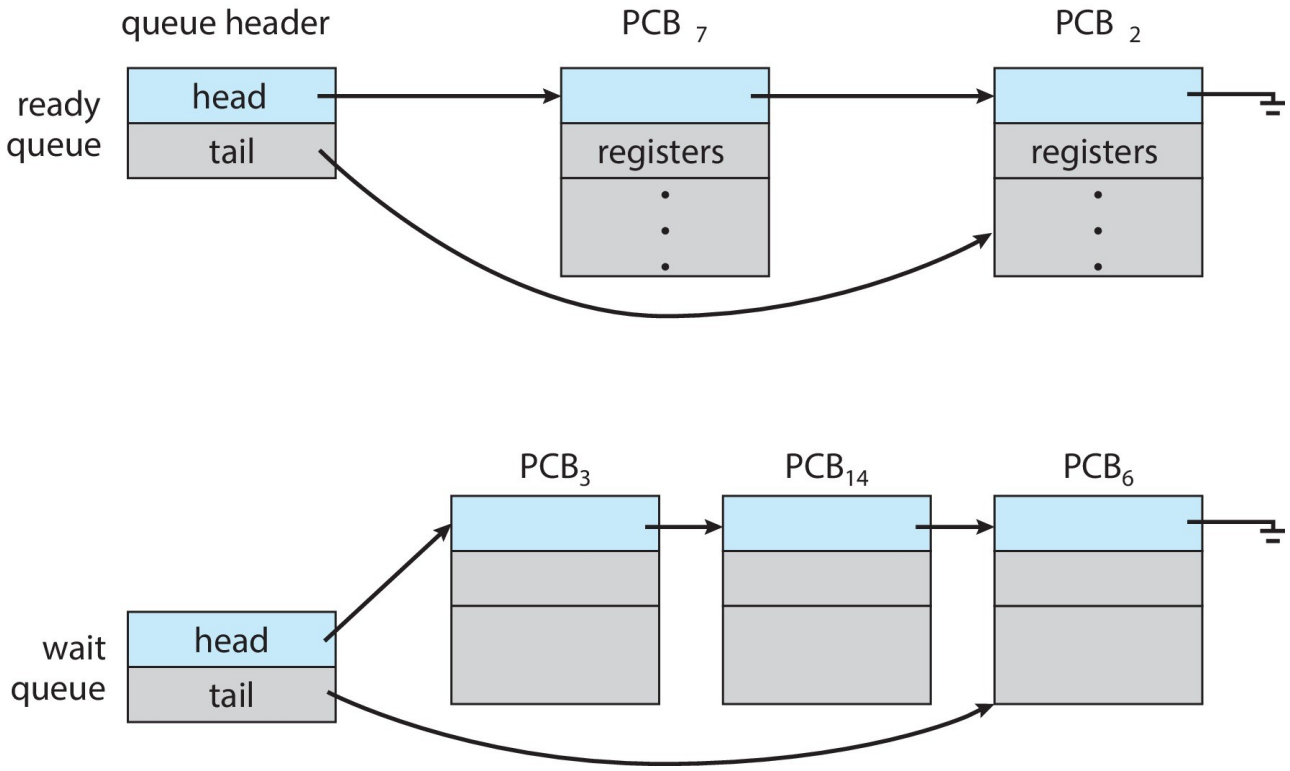
- The OS maintains **scheduling queues** of processes, e.g.

  Ready queue – set of all processes residing in main memory, ready and waiting to execute

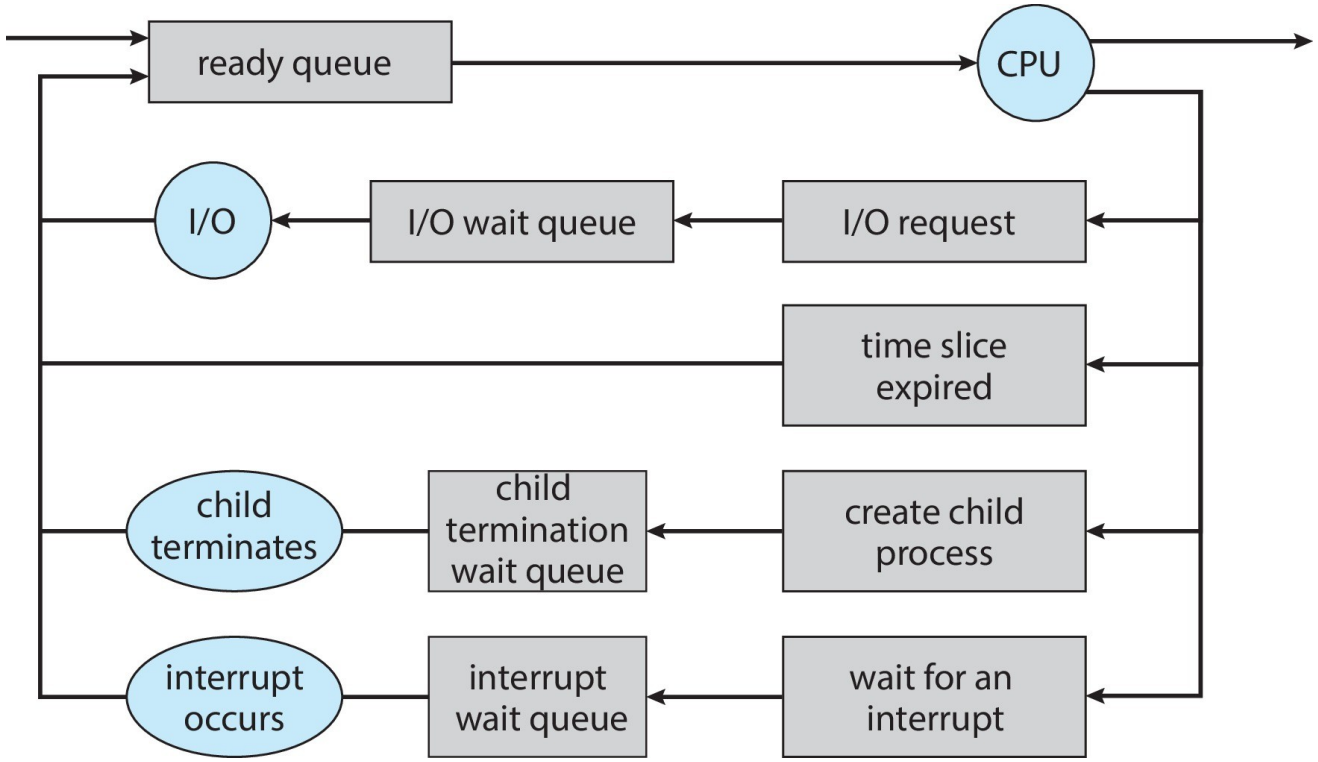
  Wait queues – set of processes waiting for an event (i.e., I/O)

  Processes migrate dynamically among the various queues

Ready and Wait Queues

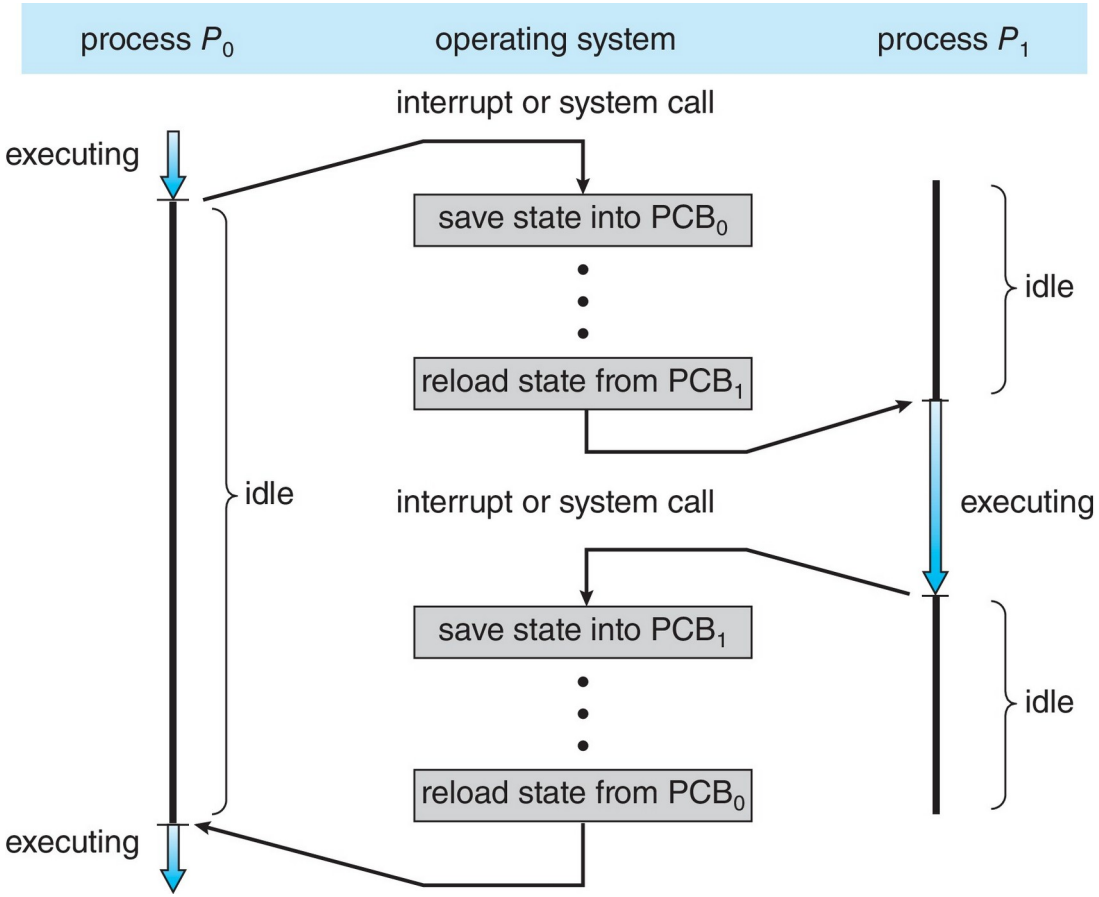


Representation of Process Scheduling



CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.



Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**

  Necessary to allow a process to resume from the exact point where it was when its execution was interrupted

- The **Context** of a process is such state information and is stored in the PCB
- Context-switch time is **pure overhead**; the system does no useful work while switching. But it is **required for concurrent activities**

  The more complex the OS and the PCB, the longer the context switch

  Some hardware provides multiple sets of registers per CPU - multiple contexts loaded at once

  Most CPUs have dedicated instructions to load/store multiple registers in one operation, often used to make context switching. E.g. LDM/STM (Load/Store Multiple) in ARM

Operations on Processes

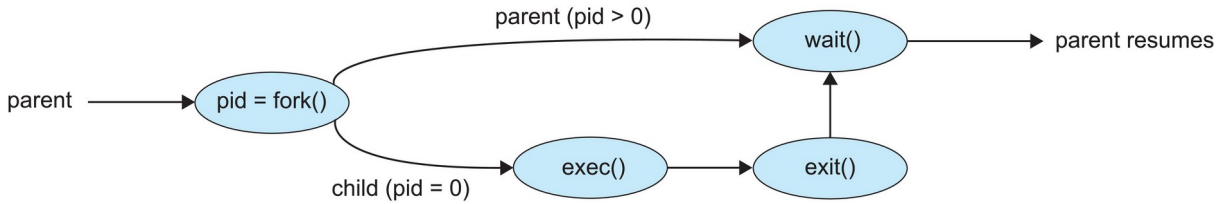
- System must provide mechanisms for:
  - Process creation
  - Process termination

Process Creation

- A **Parent** process create **children** processes, which, in turn may create other processes, forming a **tree** of processes
- Generally, a process is identified and managed via a (unique) **process identifier** (**pid**)
- Several resource sharing possibilities:
  - Parent and children **share all resources**
  - Children **share a subset** of parent's resources
  - Parent and child **share no resources**
- Also several execution options:
  - Parent and children execute **concurrently**
  - The parent **waits** until children terminate
  - And a combination: **concurrent** execution **and** at some point the parent **waits** for the children

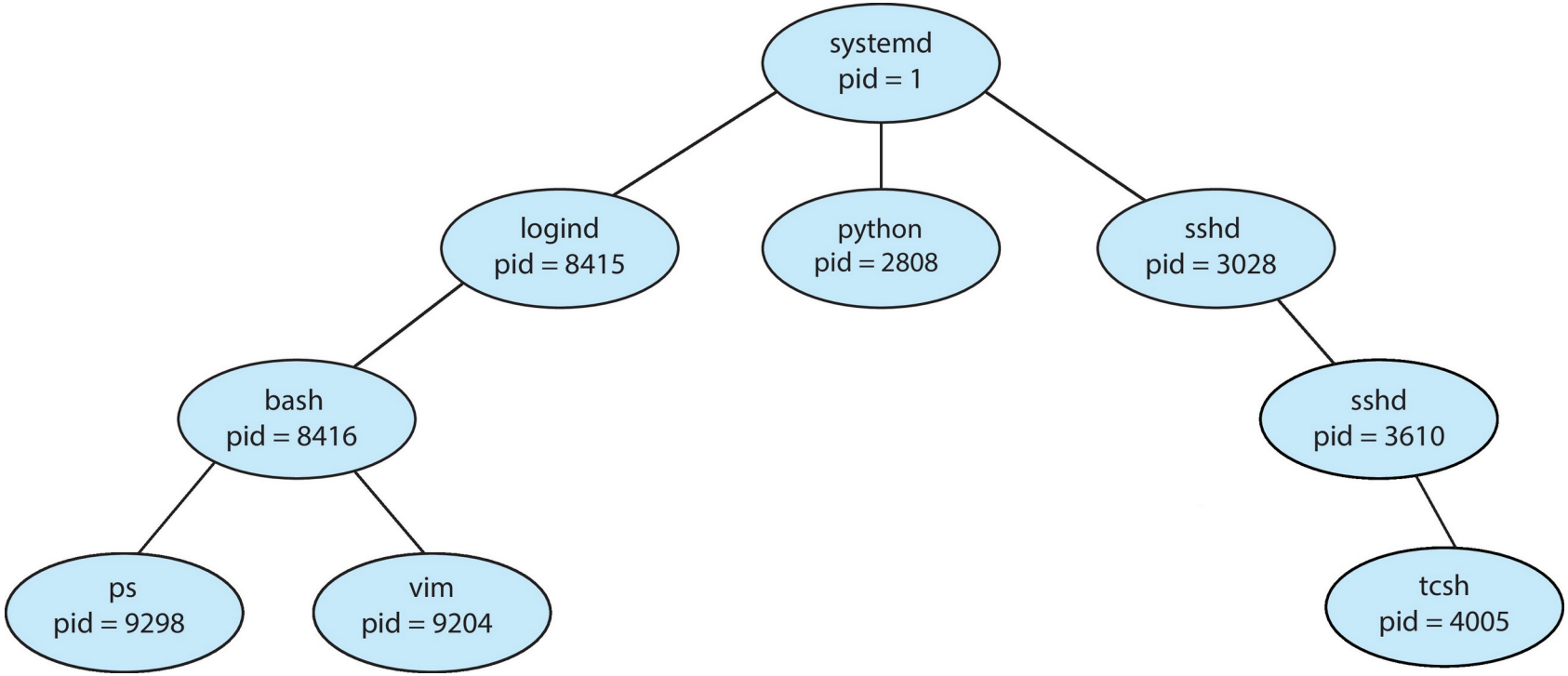
Process Creation (Cont.)

- Process creation has two steps:
  - After creation, the child receives a complete clone of the parent's address space (the exact same code, variables, open files …) - is a duplicate of parent
  - Then the OS loads the new program into the child's address space from disk, resetting the code, data, heap, and stack segments.



- UNIX/Linux examples
  - **`fork()`** system call creates new process
  - **`exec()`** system call used after a **`fork()`** to replace the process' memory space with a new program
  - Parent process calls **`wait()`** waiting for the child to terminate

A Tree of Processes in Linux



C Program Forking a Separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

Process Termination

- Process executes its last statement and then asks the operating system to delete it using the **`exit()`** system call.
  - Returns status data from child to parent (via **`wait()`**)
  - Process' resources are de-allocated by the operating system

- A parent process may terminate the execution of children processes using the **`abort()`** system call. Some reasons for doing so include:
  - Child has exceeded allocated resources
  - Task assigned to child is no longer required
  - The parent is exiting, and (possibly) the operating systems does not allow a child to continue if its parent terminates

Process Termination

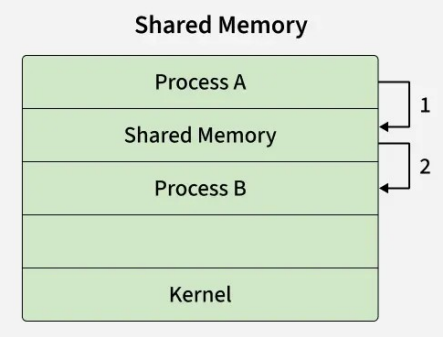
- Some operating systems do not allow child to exists if its parent has terminated. In such case if a process terminates, then all its children must also be terminated.
  - **cascading termination.** All children, grandchildren, etc., are terminated.
  - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **`wait()`** system call. The call returns status information and the pid of the terminated process

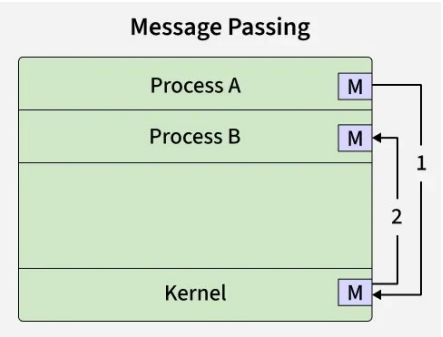
```
pid = wait(&status);
```

- If a child process terminates and the parent did not invoke **`wait()`**, the process is a **zombie** (Why?)
- If parent terminated without invoking **`wait()`**, a child process becomes an **orphan**



Inter-Process Communication - IPC





Source:https://www.geeksforgeeks.org/operating-systems/inter-process-communication-ipc/

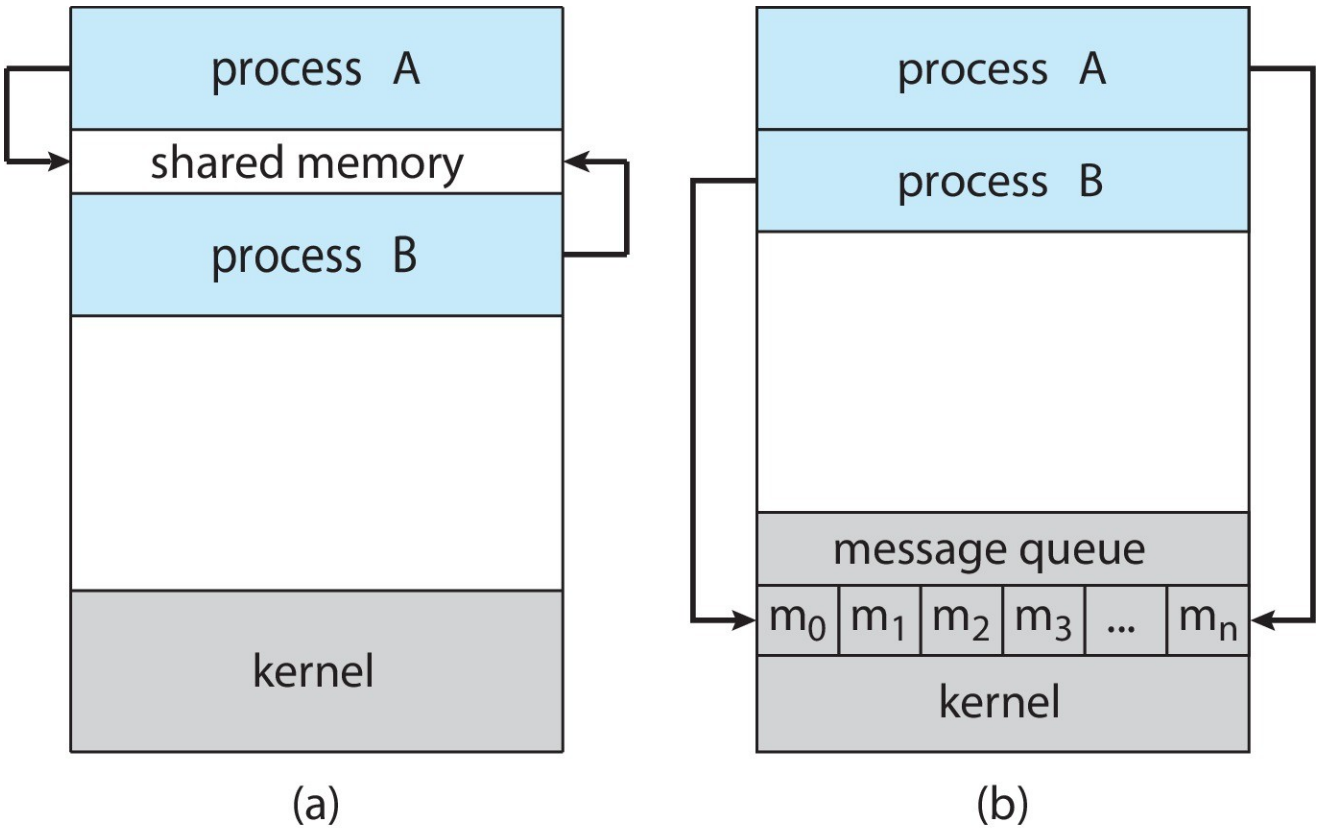
Inter-Process Communication

- Processes within a system may be ***independent*** or ***cooperating***
- **Cooperating** processes can **affect** or **be affected** by other processes, including sharing data
- Reasons for cooperating processes:
  - Information sharing
  - Computation speedup
  - Modularity
  - Convenience
- Cooperating processes need **Inter-Process Communication** (**IPC**)
- Two common models of IPC
  - **Shared memory**
  - **Message passing**

Communications Models

(a) Shared memory.

(b) Message passing.

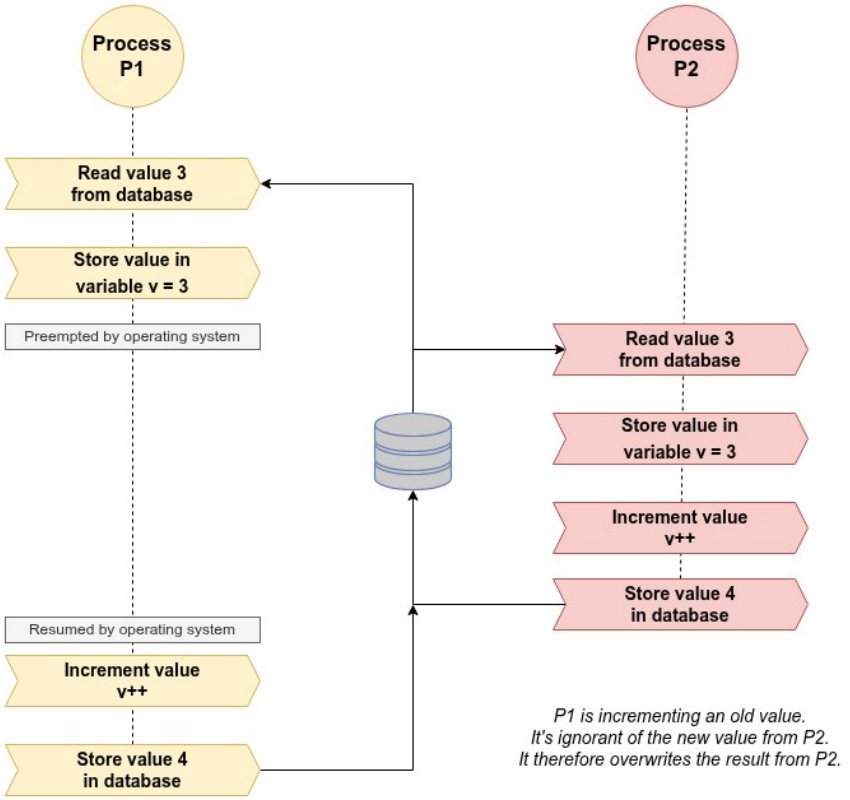


IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
  - Comparable to a global variable in a C language source file
- The **communication** is entirely under the **control of the processes**, not the operating system.
- Major issue is to provide mechanisms to allow the processes to **synchronize** their actions **when they access shared memory**.

  Concurrent access can turn data inconsistent or computational results wrong

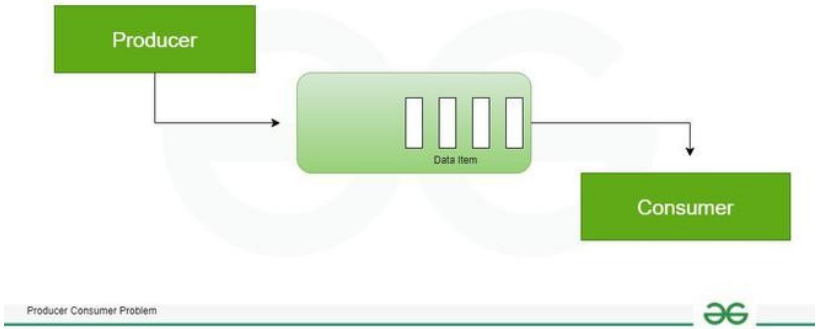
- Synchronization is discussed in detail in future classes



Illustrating software race condition. Source: Devopedia 2020.

Producer-Consumer model

- Paradigm for cooperating processes:
  - ***Producer*** process **produces** information that is **consumed by a *Consumer*** process



Source: https://www.geeksforgeeks.org/c/producer-consumer-problem-in-c/

- Two main variations:
  - **Unbounded-buffer** places no practical limit on the size of the buffer:
    - Producer never waits
    - Consumer waits if there is no buffer to consume
  - **Bounded-buffer** assumes that there is a limited buffer size
    - Producer may have to wait if all buffers are full
    - Consumer waits if there is no buffer to consume

Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
  . . .
} item_t;

item_t buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Buffer and two indexes (in/out, head/tail, …) that form a FIFO
- Solution is correct, but can only use **`BUFFER_SIZE-1`** elements.

  Why?

- It can be fixed with an additional variable (e.g. an element counter)

  Be careful in that case - race condition

Producer Process – Shared Memory

Producing one item:

```
item_t next_produced;

while (true) {
   /* produce an item in next produced */
   ...
    next_produced = xxxxxx;
   ...
   while (((in + 1) % BUFFER_SIZE) == out)
          ; /* do nothing */
   buffer[in] = next_produced;
   in = (in + 1) % BUFFER_SIZE;
}
```

Consumer Process – Shared Memory

Consuming one item:

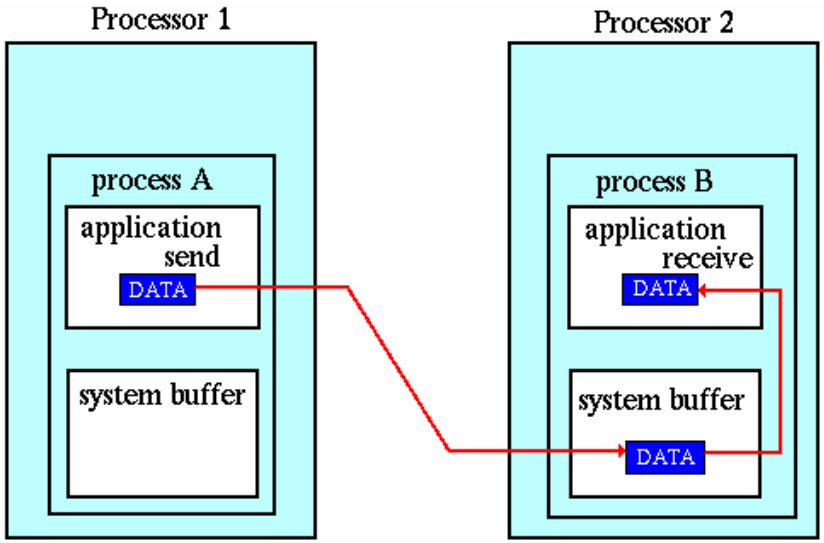
```
item_t next_consumed;

while (true) {
        while (in == out)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        /* consume the item in next consumed */
         result=f(next_consumed, ...);
}
```

IPC – Message Passing

- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
  - **send**(*message*)
  - **receive**(*message*)
- The *message* size is either fixed or variable



Path of a message buffered at the receiving process

Message Passing (Cont.)

- If processes *P* and *Q* wish to communicate, they need to:
  - Establish a ***communication link*** between them
  - **Exchange messages** via send/receive
- Implementation issues:
  - How are links established?
  - Can a link be associated with more than two processes?
  - How many links can there be between every pair of communicating processes?
  - What is the capacity of a link?
  - Is the size of a message that the link can accommodate fixed or variable?
  - Is a link unidirectional or bi-directional?

Implementation of Communication Link

- Physical:
  - Shared memory
  - Hardware bus
  - Network

- Logical:
  - Direct (processes “know” each other) or indirect (otherwise)
  - Synchronous or asynchronous
  - Automatic or explicit buffering

Direct Communication

- Processes must name each other explicitly:
  - **`send`** (*P, message*) – send a message to process P
  - **`receive`**(*Q, message*) – receive a message from process Q

- Properties of the communication link:
  - Links are established automatically
  - A link is associated with exactly one pair of communicating processes
  - Between each pair there exists exactly one link
  - The link may be unidirectional, but is usually bi-directional

Indirect Communication

- Messages are **directed and received** from **mailboxes** (also referred to as **ports**)
  - Each mailbox has a unique id
  - Processes can communicate only if they share a mailbox

- Properties of communication link
  - Link established only if processes share a common mailbox
  - A link may be associated with many processes
  - Each pair of processes may share several communication links
  - Link may be unidirectional or bi-directional

Indirect Communication (Cont.)

- Operations
  - **Create** a new mailbox/port (beginning - set up phase)
  - **Send** and **receive** messages through mailbox (during - use)
  - **Delete** a mailbox (at the end – clean up phase)

- Primitives are defined as:
  - **`send`**(*A, message*) – send a message to mailbox A
  - **`receive`**(*A, message*) – receive a message from mailbox A

Indirect Communication (Cont.)

- Mailbox sharing
  - P_1, P_2, and P_3 share mailbox A
  - P_1 sends; P_2 and P_3 receive
  - Who gets the message?

- Solutions
  - Allow a link to be associated with at most two processes
  - Allow only one process at a time to execute a receive operation
  - Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
  - **Blocking send** -- the sender is blocked until the message is received
  - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
  - **Non-blocking send** -- the sender sends the message and continues
  - **Non-blocking receive** -- the receiver receives:
    - A valid message, or
    - Null message/error code
- Different combinations are possible
  - If both send and receive are blocking, we have a **rendezvous**

Producer-Consumer: Message Passing

- Producer

```
message_t next_produced;
while (true){
  /* produce an item in next_produced */
  next_produced=f(…);
  send(A,next_produced);
}
```

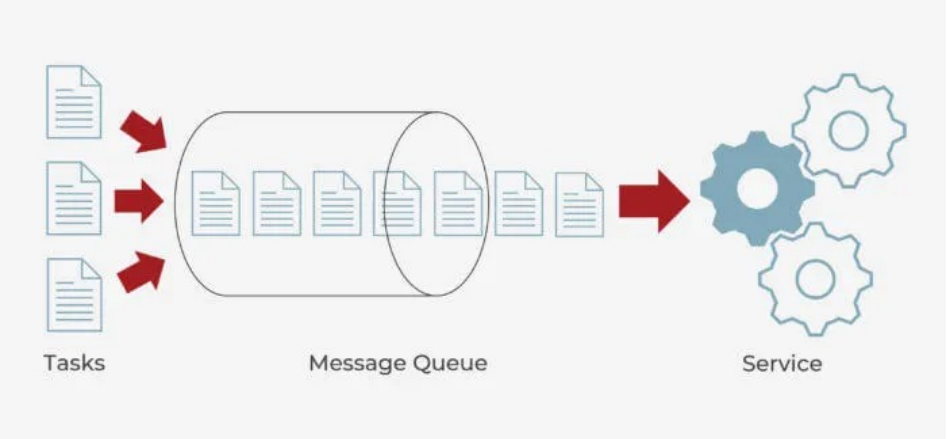
- Consumer

```
message_t next_consumed;
while (true) {
     receive(A, next_consumed)

   /* consume the item in next_consumed */
    result=f(next_consumed, …);
}
```

Link capacity - buffering

- Queue of messages attached to the link.
- Implemented in one of three ways
  1. **Zero capacity** – no messages are queued on a link.
     Sender must wait for receiver (rendezvous)
  2. **Bounded capacity** – finite length of *n* messages
     Sender must wait if link full
  3. **Unbounded capacity** – infinite length (*)
     Sender never waits

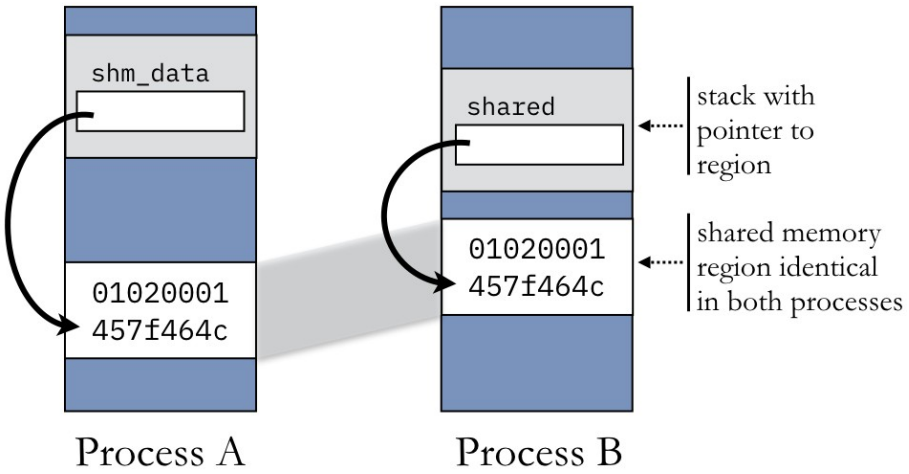


(*) In practice limited by available system resources

Example of IPC Systems - POSIX

- **POSIX Shared Memory**
  - Create and open a shared memory (shmem) segment
    `shm_fd = shm_open(name, O CREAT | O RDWR, 0666);`
  - Can also be used to open an existing shmem segment
  - Set the size of the object

    `ftruncate(shm_fd, 4096);`
  - Use `mmap()` to memory-map a file pointer to the shmem object
  - Reading and writing to shared memory is done by using the pointer returned by `mmap()`.



IPC POSIX - illustration

```
int main()
{
/* the size (in bytes) of shared memory object */
const int SIZE = 4096;
/* name of the shared memory object */
const char *name = "OS";
/* strings written to shared memory */
const char *message_0 = "Hello";
const char *message_1 = "World!";

/* shared memory file descriptor */
int shm_fd;
/* pointer to shared memory obect */
void *ptr;
```

"/OS"
For portable use a shmem object should be named in the form "/somename"

null-terminated string of up to 255 characters consisting of an initial slash, followed by one or more characters, none of which are slashes.

```
/* create the shared memory object */
shm_fd = shm_open(name, O_CREAT | O_RDWR, 0666);
```

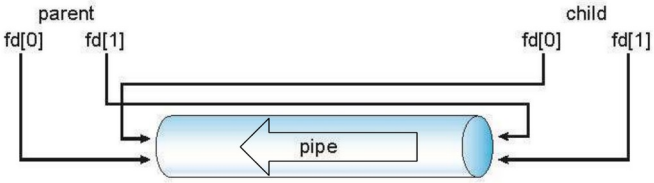
```
/* configure the size of the shared memory object */
ftruncate(shm_fd, SIZE);
```

```
/* memory map the shared memory object */
ptr = mmap(0, SIZE, PROT_WRITE, MAP_SHARED, shm_fd, 0);
```

```
/* write to the shared memory object */
sprintf(ptr,"%s",message_0);
```

Access via pointer – can be read, written ...

Pipes



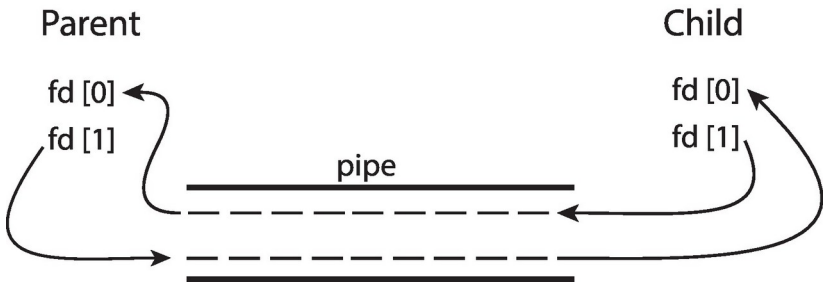
- Acts as a conduit allowing two processes to communicate
- Issues:
  - Is communication unidirectional or bidirectional?
  - In the case of two-way communication, is it half or full-duplex?
  - Must there exist a relationship (i.e., ***parent-child***) between the communicating processes?
  - Can the pipes be used over a network?

- **Ordinary pipes** – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.

- **Named pipes** – can be accessed without a parent-child relationship.

Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



- Windows calls these **anonymous pipes**

Named Pipes

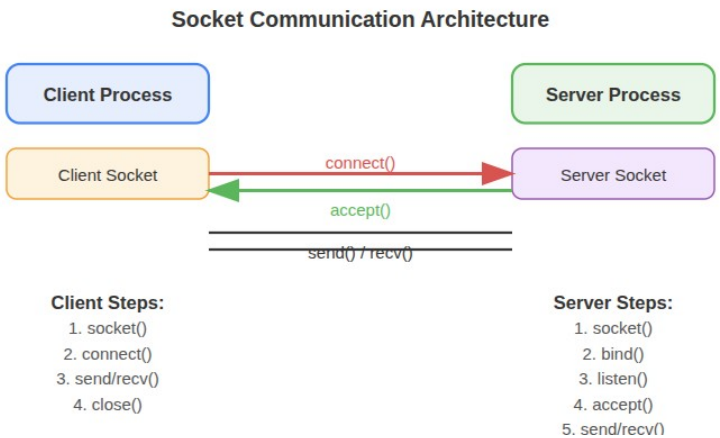
- **Named Pipes are more powerful** than ordinary pipes
  - Communication is **bidirectional**
  - **No parent-child** relationship is necessary between the communicating processes
  - **Several processes** can use the named pipe for communication

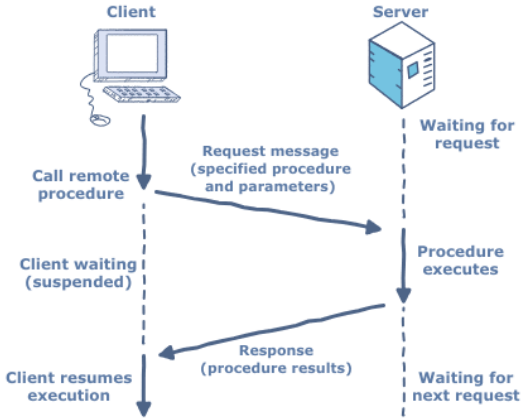
- Provided on both Linux/Unix and Windows systems
  - Linux/Unix: also called First-In-First-Out (FIFO)
    Create a special file on the filesystem that can be accessed by any process with appropriate permissions.



Communications in Client-Server Systems

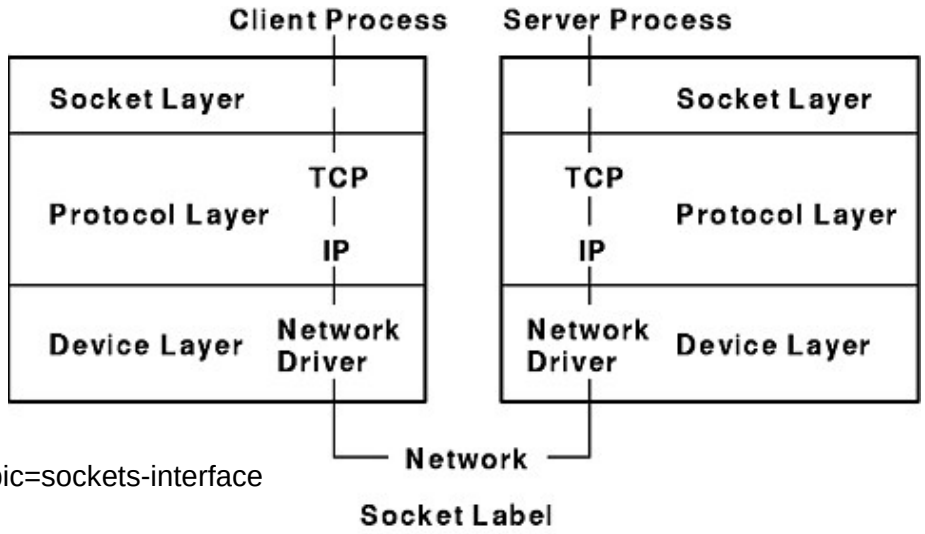
- Sockets
- Remote Procedure Calls





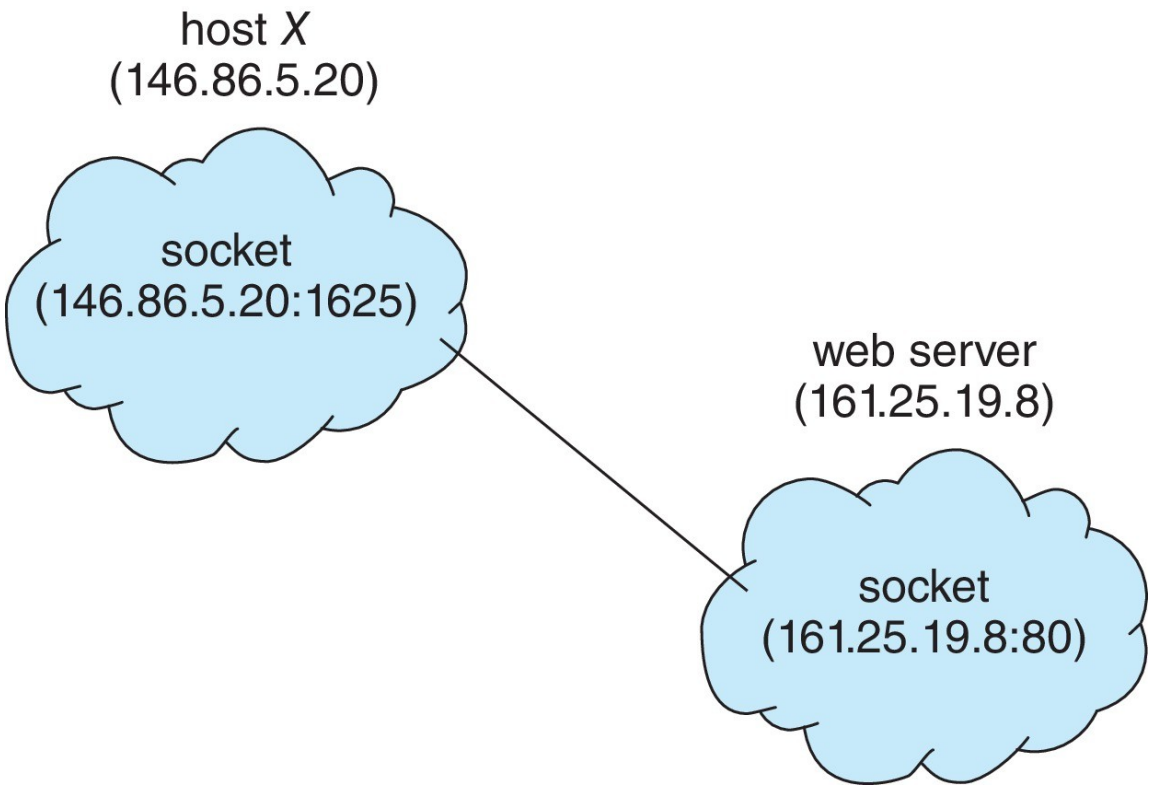
Sockets

- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
  - socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are ***well known***, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running



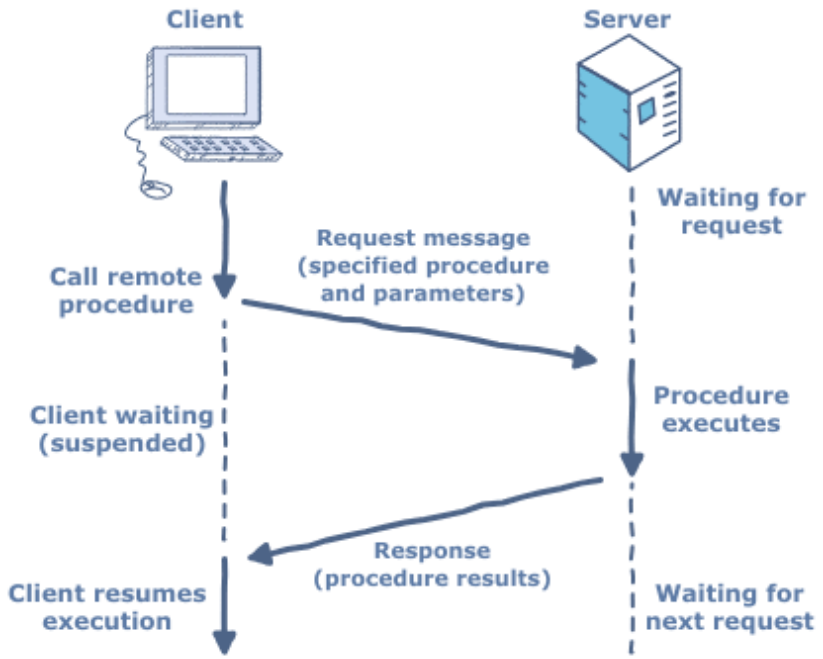
https://www.ibm.com/docs/en/aix/7.2.0?topic=sockets-interface

Socket Communication



Remote Procedure Calls

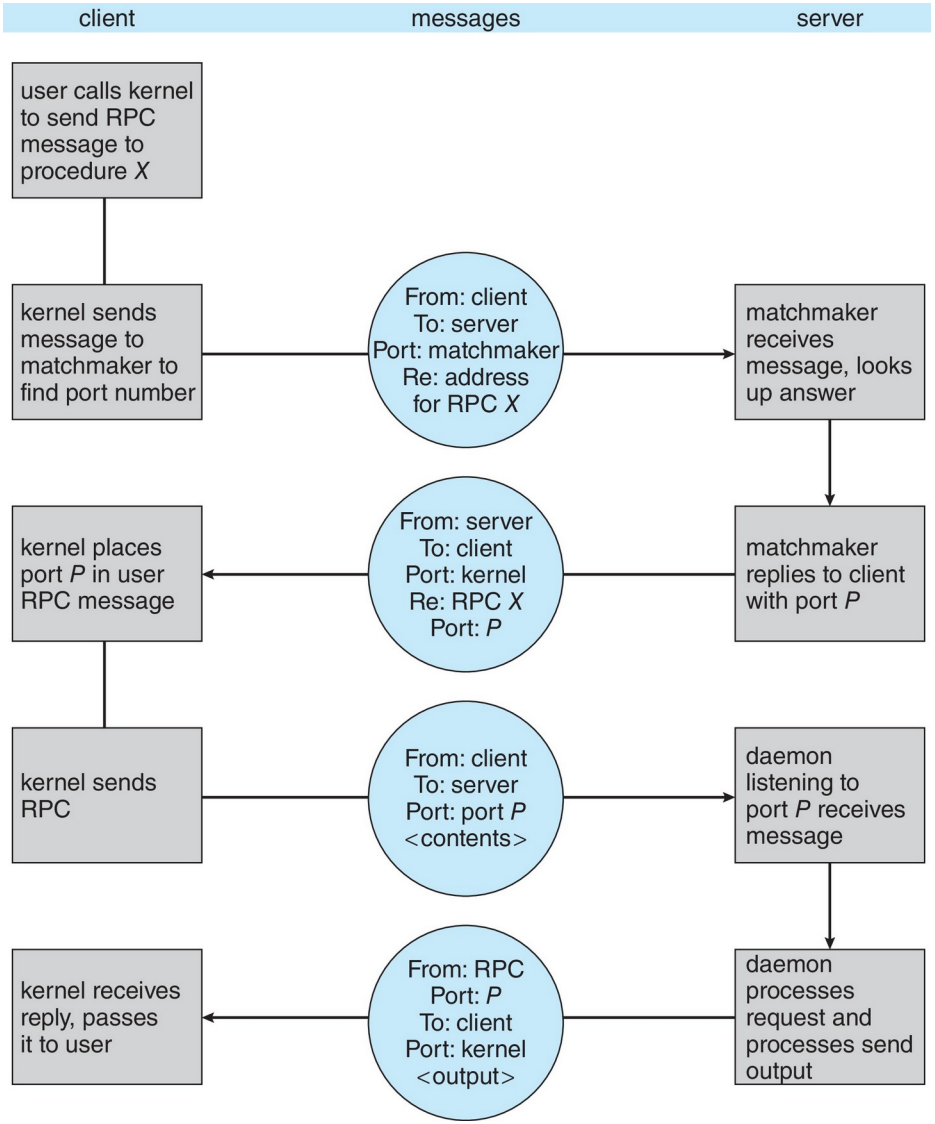
- Remote Procedure Call (RPC) abstracts procedure calls between processes on networked systems
  - Again uses ports for service differentiation
- **Stubs** – client-side proxy for the actual procedure on the server
- The client-side stub locates the server and **marshalls** the parameters
- The server-side stub receives this message, unpacks the marshalled parameters, and performs the procedure on the server
- On Windows, stub code is generated from a specification written in **Microsoft Interface Definition Language** (**MIDL**)
  - Developers do not manually write the (complex) network communication code



Remote Procedure Calls (Cont.)

- Data representation handled via **External Data Representation** (**XDL**) format to account for different architectures
  - **Big-endian** and **little-endian**
- Remote communication has more failure scenarios than local
  - Messages can be delivered ***exactly once*** rather than ***at most once***
- OS typically provides a rendezvous (or **matchmaker**) service to connect client and server

Execution of RPC





- Questions?

Acknowledgment:

This presentation is based on the companion materials provided with the book

Operating System Concepts – 10h Edition, by Silberschatz, Galvin and Gagne